Table of Contents

1. Annotation 3
2. Introduction 3
3. Task and Dataset 4
    3.1. Voice Spoofing and Logical Access 4
    3.2. ASVspoof 2019 Dataset 4
4. Methodology 5
    4.1. Input Features and Data Preparation 5
    4.2. LCNN Model Architecture 5
    4.3. Loss Function 6
    4.4. EER Metric 6
5. Experimental Setup 6
    5.1. Training Configuration 6
    5.2. Training 7
    5.3. Initial Model Verification: One-Batch Test 7
6. Results 7
7. Conclusion 8
8. Calendar Schedule 9
9. The List of Sources 10
10. Appendixes 11

1. Annotation

Automatic speaker verification is increasingly used for biometric authentication. The development of speech synthesis and voice conversion technologies creates a real threat of bypassing such systems using spoofed speech, which makes the task of automatic spoofing attack detection practically important. During my internship, I studied the basics of deep learning, modern neural network architectures, and approaches to solving audio recording classification tasks using deep learning methods. I applied the knowledge I gained to the practical homework task: I implemented and trained a countermeasure system for detecting synthesized speech on the Logical Access part of the ASVspoof 2019 dataset. At the end of the work, I analysed the obtained results and evaluated the model performance. As a result of training, the system achieved EER = 4.4735% on the evaluation set, which confirms that the selected approach worked.

2. Introduction

At present, automatic speaker verification is one of the common technologies used for biometric authentication. Such systems make it possible to confirm a user's identity based on the characteristics of their speech. They are used in banking services, access control systems, telephone services, and other information systems. The development of speech synthesis (text-to-speech, TTS) and voice conversion (voice conversion, VC) technologies has threatened the reliability of such systems: modern algorithms can generate speech that is almost indistinguishable in sound from real speech. Attacks of this kind are commonly called voice spoofing or deepfake speech. Since an ASV system in this situation cannot physically distinguish live speech from synthesized speech, incorrectly accepting a spoofing attack as a real user directly threatens the security of biometric systems. To counter such attacks, separate detector models, called countermeasure systems (CM), are developed. They analyse a recording and assign it to one of two classes: bonafide (real speech) or spoof (fake speech).

The aim of the internship was to study the basics of deep learning and writing code for neural networks, as well as to apply the knowledge gained to a practical task.

To achieve this aim, I was given the following tasks:

1. Study the basics of deep learning.

2. Learn how to train and use neural networks in code.

3. Apply the knowledge gained in practice by solving the homework task.

The following main topics were covered during the internship:

1. The basic concepts of deep learning.

2. Advanced topics: multimodal networks, large models, and audio processing.

I applied the knowledge gained in practice while completing the homework task, which consisted of implementing and training a countermeasure system for detecting spoofed speech on the Logical Access part of the ASVspoof 2019 dataset.

During the work, I analysed the structure of the dataset, implemented audio file loading, prepared spectral features, and implemented the LCNN architecture. After that, I trained the model and evaluated its quality.

3. Task and Dataset

3.1. Voice Spoofing and Logical Access

Voice spoofing is the creation or use of a voice signal that imitates a person's natural speech. Such an attack may be aimed at bypassing an automatic speaker verification system. In the Logical Access scenario, spoofed speech is generated digitally. In this case, the attacker does not need to reproduce the audio recording through a loudspeaker. The speech can be created using a synthesis algorithm or converted from another person's recording.

The main types of Logical Access attacks include:

1. speech synthesis using Text-to-Speech;

2. voice conversion using Voice Conversion;

3. other methods of digital generation or modification of a speech signal.

In this task, two classes are used:

1. bonafide — natural speech of a real person;

2. spoof — synthesized or converted speech.

Thus, the task is reduced to the binary classification of audio files. The model must determine whether the input signal belongs to natural speech or is spoofed.

3.2. ASVspoof 2019 Dataset

To complete the task, I used the Logical Access part of the ASVspoof 2019 dataset. The set was created to study methods of protecting automatic speaker verification systems from spoofing attacks. The audio files are presented in FLAC format. For each file, there is a line in the protocol containing information about the speaker, the file name, the spoofing system, and the class.

A special feature of ASVspoof 2019 is that the evaluation set contains unknown or modified attack variants. Therefore, the final evaluation shows how well the model generalises the learned features to new data.

After analysing the protocols, I obtained the following distribution across the LA subsets:

1. The training set contains 25,380 files, the development set contains 24,844 files, and the evaluation set contains 71,237 files.
2. The training set contains 2,580 bonafide files and 22,800 spoof files.
3. The development set contains 2,548 bonafide files and 22,296 spoof files.
4. The evaluation set contains 7,355 bonafide files and 63,882 spoof files.

A strong class imbalance can be observed in all three subsets, approximately one genuine recording for every nine spoofed recordings. I took this imbalance into account when choosing the loss function. In my implementation, I trained the model on the training subset and evaluated it, both during training and for the final metric, on the evaluation subset. Separate validation on the development subset was not used in the pipeline.

4. Methodology

4.1. Input Features and Data Preparation

As input features, I used a spectral representation of the audio signal. For this purpose, I applied the Short-Time Fourier Transform. STFT represents an audio signal as a dependence of energy on time and frequency. This format is convenient for convolutional neural networks because the spectrum can be considered as a two-dimensional matrix similar to an image.

The following parameters were used to calculate the spectrum:

- n_fft = 1724;
- win_length = 1724;
- hop_length = 130;
- Blackman window function;
- number of time frames = 600.

After calculating the spectrum, I used the logarithm of the power spectrum. To prevent the logarithm of zero from being calculated, a lower bound was applied. If the spectrum was longer than 600 frames, a random fragment of length 600 frames was selected from it during training. This made it possible to use different fragments of the same file during different training iterations. If the spectrum was shorter than 600 frames, it was extended using cyclic repetition. This ensured that all input objects had the same size. I did not use additional feature normalisation or voice activity detection. This approach corresponds to the description of the FFT-LCNN system in the paper “STC Antispoofing Systems for the ASVspoof2019 Challenge”.

4.2. LCNN Model Architecture

For audio file classification, I implemented the Light Convolutional Neural Network architecture. This architecture is used to extract informative features from the input signal and contains special MFM activations. The main feature of LCNN is the Max-Feature-Map. This operation receives several groups of channels and selects the maximum value between corresponding channels.

MFM can be represented in a simplified form: $MFM(x) = \max(x_1, x_2)$, where x_1 and x_2 are two groups of features. Using MFM allows the model to select the strongest features and reduces the number of channels after the operation.

The general structure of the model is as follows:

- Input 1 × 863 × 600

- Conv 5×5 + MFM + MaxPool
- Conv 1×1 + MFM + BatchNorm
- Conv 3×3 + MFM + MaxPool + BatchNorm
- Conv 1×1 + MFM + BatchNorm
- Conv 3×3 + MFM + MaxPool
- Conv 1×1 + MFM + BatchNorm
- Conv 3×3 + MFM + BatchNorm
- Conv 1×1 + MFM + BatchNorm
- Conv 3×3 + MFM + MaxPool
- Flatten + Linear + MFM
- Dropout + BatchNorm
- Linear(2)

After the convolutional part, the features were converted into a one-dimensional vector. Then they were passed to a fully connected layer with a size of 160. After this layer, MFM activation was used, reducing the feature vector size to 80. Dropout with a value of 0.75 was placed before the final Batch Normalization. I used Kaiming initialization to initialize the weights of the convolutional and fully connected layers.

4.3. Loss Function

For model training, I used Cross-Entropy Loss with label smoothing. Cross-Entropy is suitable for the binary classification task because the model produces two logits: one for the bonafide class and one for the spoof class. The Speech Technology Center paper used angular margin based softmax. However, a comparative study notes that Cross-Entropy and simpler classification functions can also show good results for LCNN systems. In addition, margin-based functions require additional hyperparameter tuning. Therefore, I chose Cross-Entropy as a simpler loss function. It allows the model to be trained without adding an additional angular margin mechanism. As additional regularisation, I used label smoothing with a value of 0.1. It reduces the excessive confidence of the model and can reduce the risk of overfitting.

4.4. EER Metric

As the main quality metric, I used Equal Error Rate (EER). During classification, two main errors can occur:

1. accepting a spoof file as bonafide;
2. accepting a bonafide file as spoof.

False Acceptance Rate shows the proportion of spoof files that were mistakenly accepted as natural speech. False Rejection Rate shows the proportion of bonafide files that were mistakenly rejected. EER is determined at the point where these two errors are approximately equal:

EER = FAR = FRR

The lower the EER value, the better the model distinguishes natural speech from spoofed speech.

5. Experimental Setup

5.1. Training Configuration

For model training, I used the following parameters:

- optimizer = Adam;
- initial learning rate = $3 \cdot 10^{-4}$;
- $\beta_1 = 0.9$;
- $\beta_2 = 0.999$;
- $\varepsilon = 10^{-8}$;
- batch size = 8;
- dropout = 0.75;
- number of epochs = 13;
- seed = 0;
- number of workers = 4.

StepLR was used to change the learning rate. After 10 epochs, the learning rate was reduced by half. Thus, after the tenth epoch, it changed from $3 \cdot 10^{-4}$ to $1.5 \cdot 10^{-4}$. The selected training parameters were based on the paper "A Comparative Study on Recent Neural Spoofing Countermeasures for Synthetic Speech Detection".

5.2. Training

After preparing the data, I trained the LCNN model on the training set. At each iteration, the model received a batch of spectral features and calculated logits for two classes, after which Cross-Entropy Loss was calculated. After each epoch, I switched the model to evaluation mode and calculated the average loss and EER.

The training set contained 25,380 files. With a batch size of 8 and drop_last=True, one epoch contained 3,172 batches. The full training lasted 13 epochs.

During training, I saved two types of checkpoint:

1. the last checkpoint;

2. the best checkpoint with the minimum EER.

For the final result, I used the best checkpoint because the EER changed non-linearly and the last checkpoint did not necessarily correspond to the best result.

5.3. Initial Model Verification: One-Batch Test

Before full training, I tested the model on one fixed batch. This test was needed to check the main stages of the pipeline. For the test, I selected a batch of eight examples containing both classes. The model was trained on this batch for 200 epochs. After the successful One-Batch Test, I proceeded to full model training on the training set.

6. Results

The attached graph in the Appendixes contains two panels: the left panel shows the train loss and eval loss curves by epoch, and the right panel shows EER on the evaluation subset by epoch. Train loss behaves as expected: after the first 2–3 epochs, it quickly reaches a low plateau of about 0.22 and then almost does not change. The model adapts to the training data rather quickly, while the loss does not fall completely to zero because label smoothing intentionally prevents the logits from “running away” to infinity.

The EER curve on the evaluation subset fluctuates in a similar way: there are noticeable spikes, particularly at the beginning of training, in epochs 2–3, and later around epoch 6. However, EER decreases globally as training progresses. It decreases not linearly, but in jumps, with local deteriorations between epochs. Because of this non-linearity, the final model was not taken from the last, thirteenth epoch, but from the checkpoint that showed the lowest EER during the whole training: since the model quality fluctuates rather than decreases monotonically, the last epoch by number is not necessarily the best in fact, which is what happened in my case.

The best checkpoint was obtained at the tenth epoch. After loading it and running a full evaluation on the evaluation set, I obtained the final EER value of 4.4735%.

7. Conclusion

During the internship, I studied the basics of deep learning and applied the knowledge I gained to the task of detecting synthesized speech. I analysed the structure of the ASVspoof 2019 dataset and used its Logical Access part. For each audio file, I prepared a spectral representation based on STFT. The logarithm of the power spectrum with a fixed size of 863 by 600 was used as input data. For classification, I implemented an LCNN architecture with MFM activations, convolutional layers, Batch Normalization, and Dropout. Cross-Entropy Loss, the Adam optimizer, and a scheduler for changing the learning rate were used for training. Before full training, I conducted a One-Batch Test. The model was able to achieve an EER of 0% on the fixed batch, which confirmed that the main pipeline worked correctly. Full training lasted 13 epochs. EER changed non-linearly, so I used the best checkpoint rather than the checkpoint from the final epoch to obtain the final result. The best EER value was 4.4735%.

As a result, I implemented a working countermeasure system for detecting spoofed speech and obtained a result comparable to the result of a separate FFT-LCNN system described in the literature.

The practical work allowed me to consolidate my knowledge of deep neural networks, learn how to work with audio data, and complete the full model development cycle: from dataset preparation to creating a file with final predictions.

9. The List of Sources

1. Kaggle. ASVspoof 2019 Dataset [Electronic resource] – URL: https://www.kaggle.com/datasets/awsaf49/asvpoof-2019-dataset (accessed: 14 July 2026).
2. Yamagishi J., Todisco M., Sahidullah M., Delgado H., Wang X., Evans N., Kinnunen T., Lee K. A., Vestman V., Nautsch A. ASVspoof 2019: Automatic Speaker Verification Spoofing and Countermeasures Challenge Evaluation Plan [Electronic resource]. 2019. – URL: https://www.asvspoof.org/asvspoof2019/asvspoof2019_evaluation_plan.pdf (accessed: 14 July 2026).
3. Lavrentyeva G., Novoselov S., Tseren A., Volkova M., Gorlanov A., Kozlov A. STC Antispoofing Systems for the ASVspoof2019 Challenge // Proceedings of Interspeech. 2019. P. 1033–1037.
4. Wang X., Yamagishi J. A Comparative Study on Recent Neural Spoofing Countermeasures for Synthetic Speech Detection [Electronic resource]. 2021. – URL: https://arxiv.org/abs/2103.11326 (accessed: 14 July 2026).

10. Appendixes

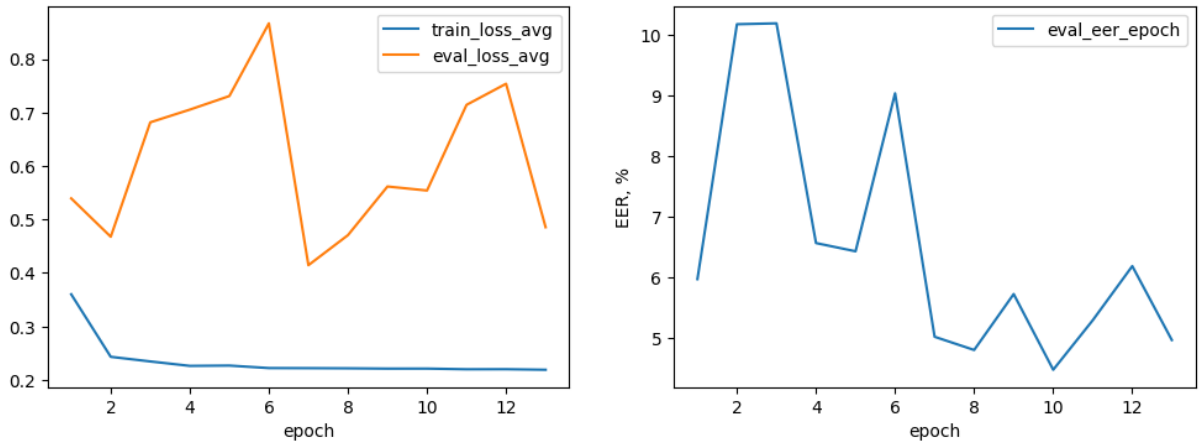


Figure 1. Training and evaluation loss and EER

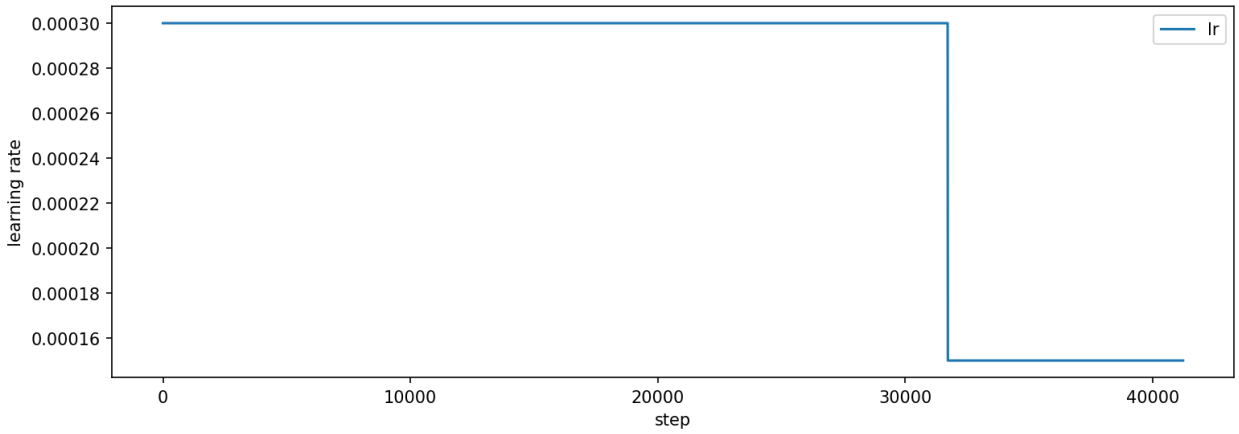


Figure 2. Learning-rate.